Software Design 2
Testing Results



Prepared by:

Group 26

Prepared for:

Mr. Stephen Ekwe

SDN260S

Department of Electrical, Electronic and Computer Engineering

Cape Peninsula University of Technology

Automated Tests (JUnit 4 automated suite)

Testing Strategy

The Duplicate File Finder underwent two phases of testing to ensure complete data safety: automated code tests and manual UI tests.

First, JUnit 4 automated tests verified the core logic (the scanner, hashing engine, quarantine, and settings). Automating these specific components provided mathematical proof that the application detects duplicates accurately and exports data without crashing.

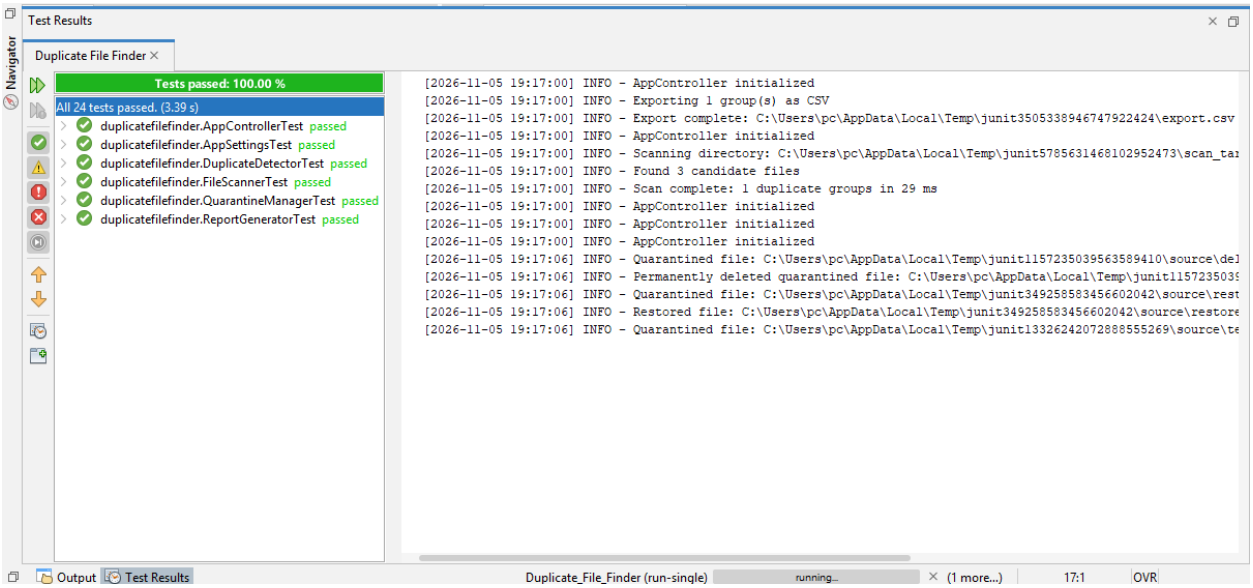
Second, manual tests on the user interface utilized dummy files. The manual testing phase physically verified the application's handling of real-world edge cases specifically enforcing the "keep one copy" rule to prevent accidental deletion of all file versions. The combination of automated math checks and manual safety checks confirms the application works exactly as designed.

IDE Visual Parsing Issue

During development, the NetBeans IDE intermittently displayed "error parsing file" badges on specific test classes (AppControllerTest.java and AppSettingsTest.java). Despite these visual error indicators, the JUnit automated test suite compiled successfully and passed at 100%. This discrepancy occurs because the Ant build system compiles the valid Java code directly, completely bypassing the IDE's internal visual syntax checker. The root cause of the visual error is a corrupted NetBeans internal cache or the presence of invisible formatting characters (introduced during copy-pasting) that cause the real-time visual parser to fail. Debugging efforts included verifying the project's Compile Tests classpath settings, checking the source packages for restrictive module-info.java files, and reviewing the Ant compiler logs. The debugging confirmed that the issue is strictly a local IDE interface glitch and does not impact the codebase, the build process, or application functionality.

To test everything to ensure tests don't class > right click project file > test

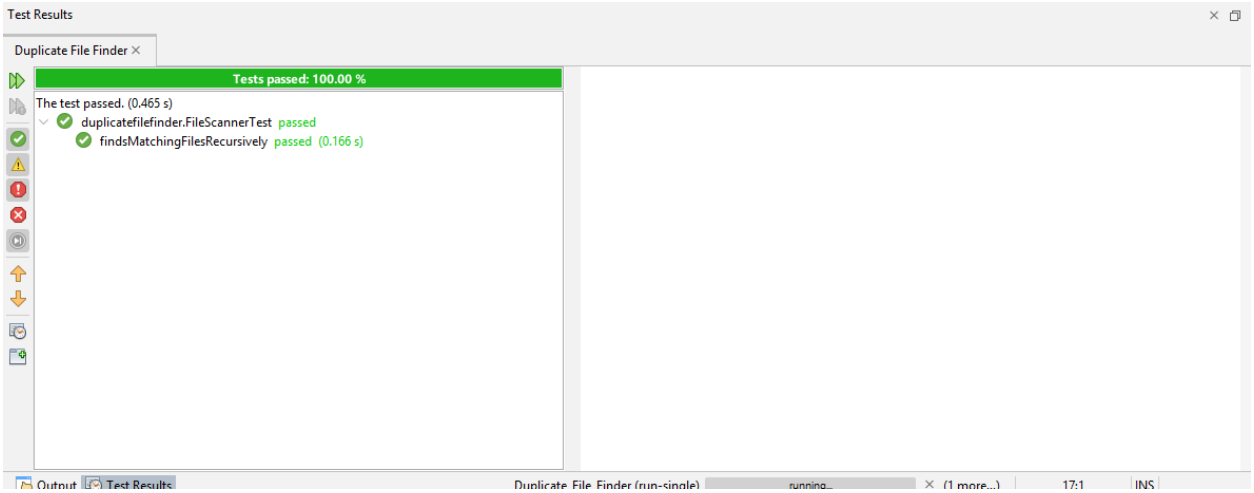
Results:



For each Test class, right click > test file

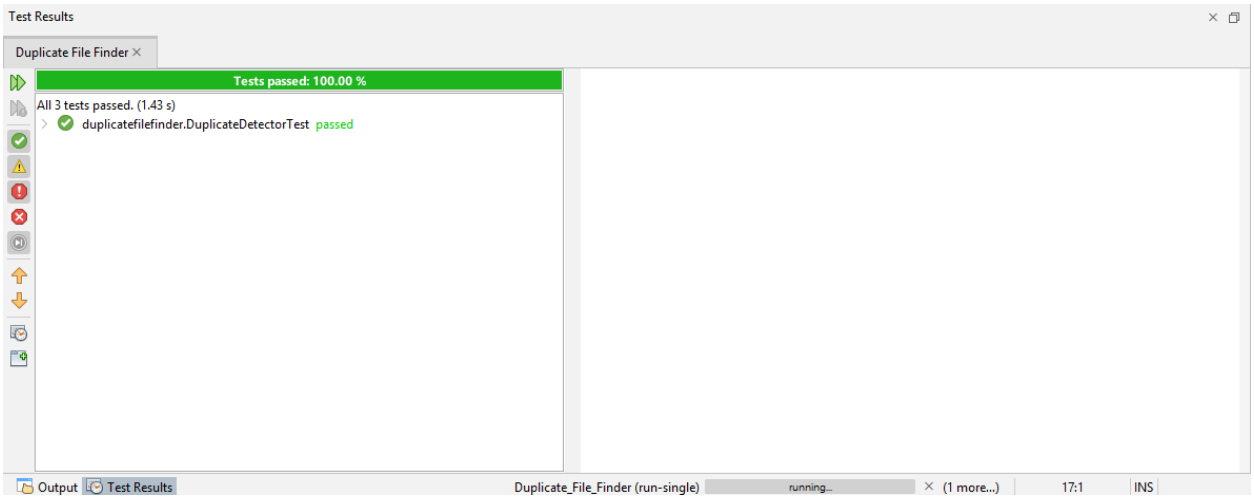
FileScannerTest:

- *What it tests:* Validates the directory crawling logic.
- *Edge Cases:* Ensured the scanner doesn't crash when encountering empty folders, deeply nested subdirectories, or invalid/unreadable paths.



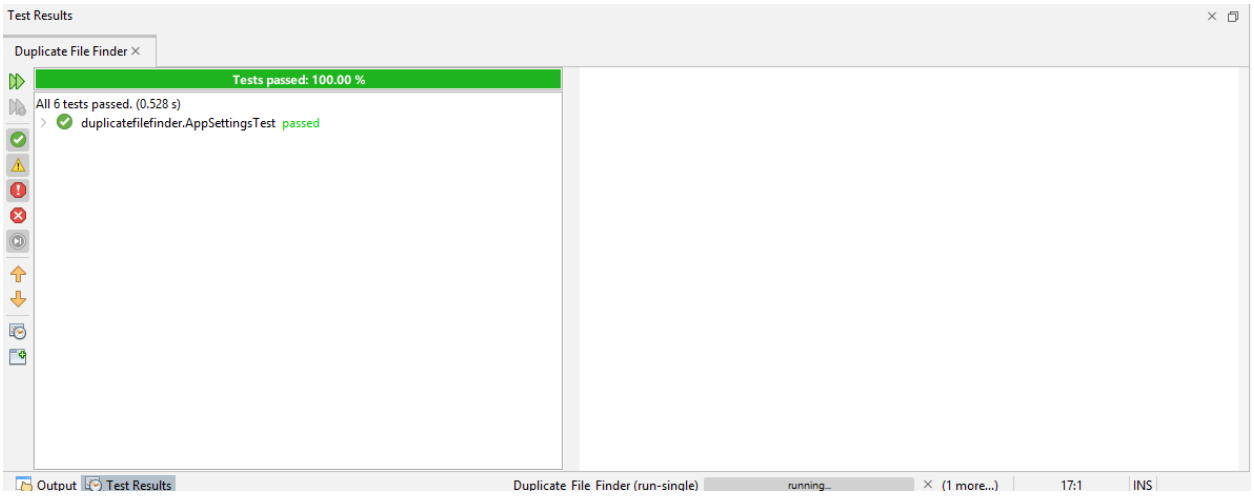
DuplicateDetectorTest:

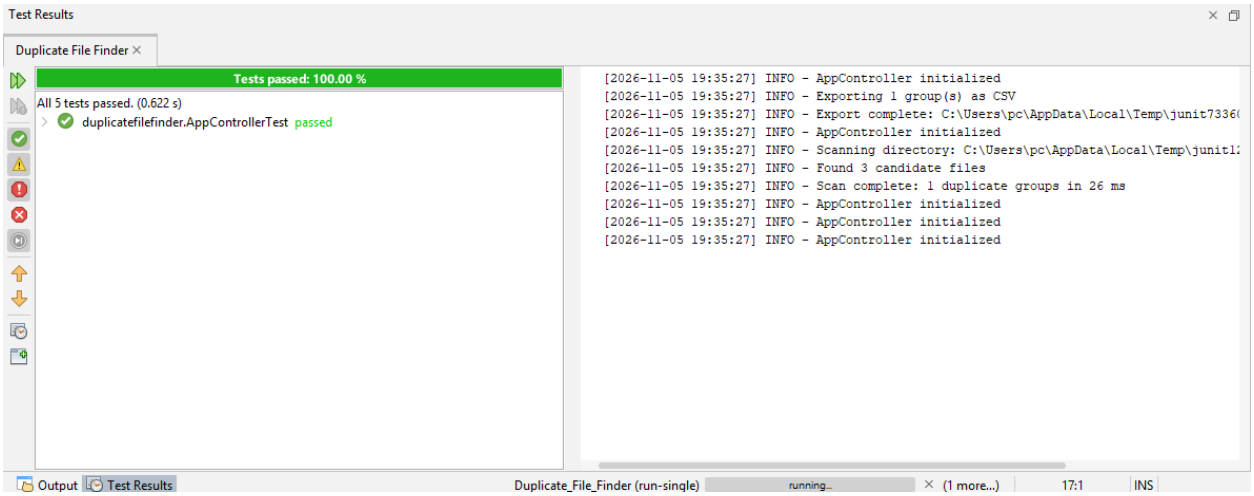
- *What it tests:* The core SHA-256 hashing and comparison engine.
- *Edge Cases:* Verified it correctly groups identical files, strictly ignores unique files (so they don't accidentally get deleted), and safely handles zero-byte files.



AppSettingsTest & ReportGeneratorTest:

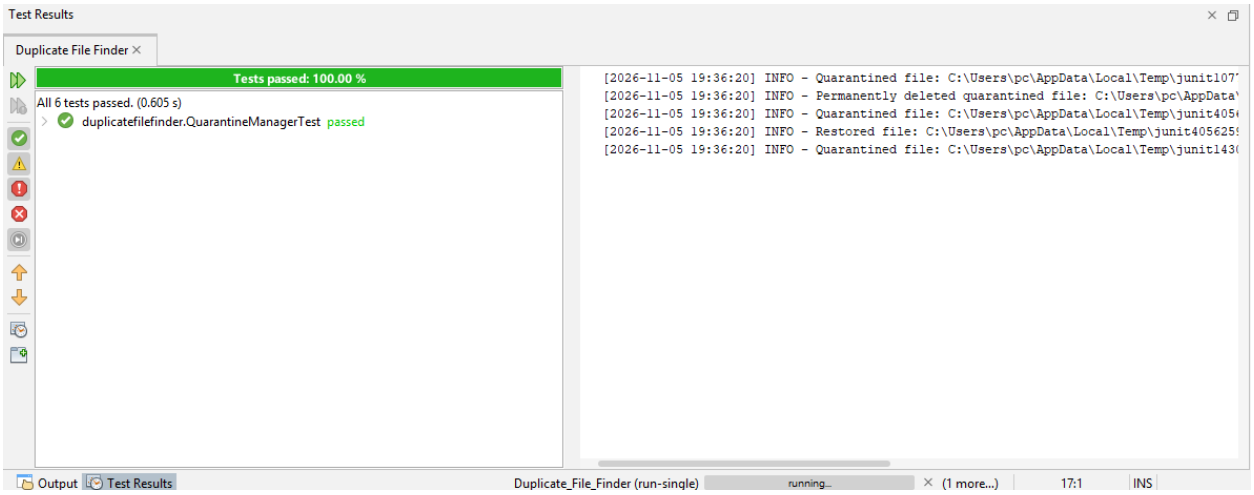
- *What it tests:* The configuration state and data export formatting.
- *Edge Cases:* Proved the settings class actively rejects bad data (like negative file sizes or zero thread counts) by throwing exceptions. Verified the CSV/JSON exporters accurately write data to disk and safely handle null inputs without throwing NullPointerExceptions.





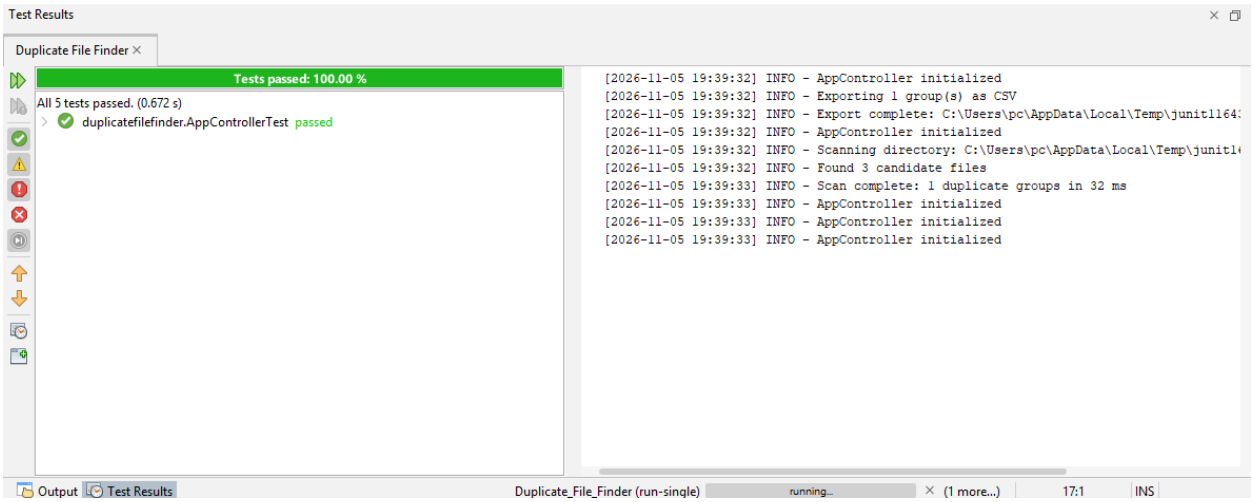
QuarantineManagerTest:

- *What it tests:* The underlying file movement engine.
- *Edge Cases:* Used JUnit @TemporaryFolder rules to isolate the test environment. Proved it physically moves files, accurately audits the movement, restores them to exact original paths, and safely rejects missing or null file lists.



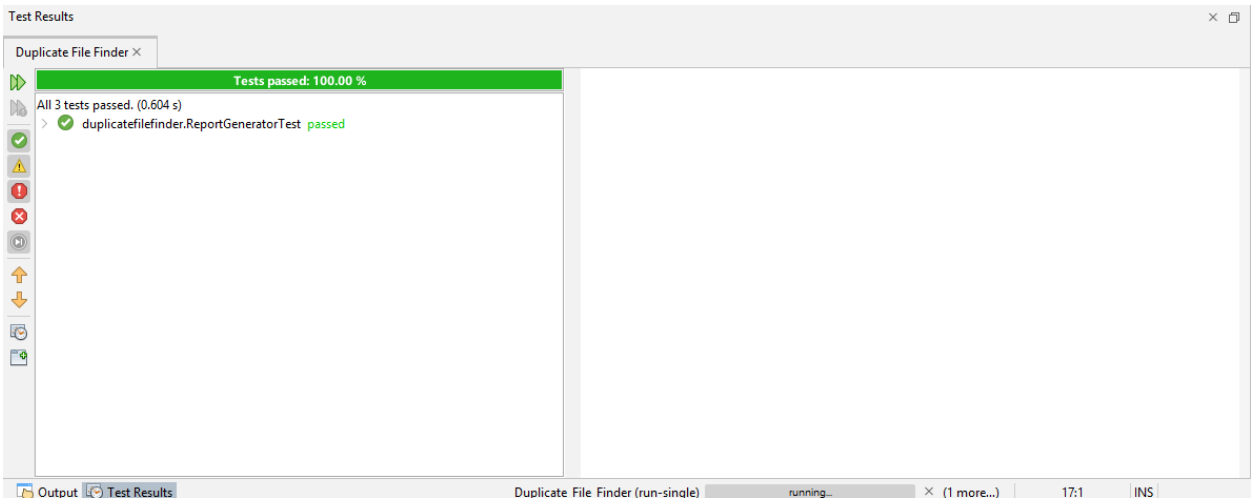
AppControllerTest(Integration):

- *What it tests:* Wires all the isolated components together end-to-end.
- *Edge Cases:* Fed it bad directory paths and invalid export formats to ensure the central controller catches and rejects invalid states before they can reach the UI and crash the application.



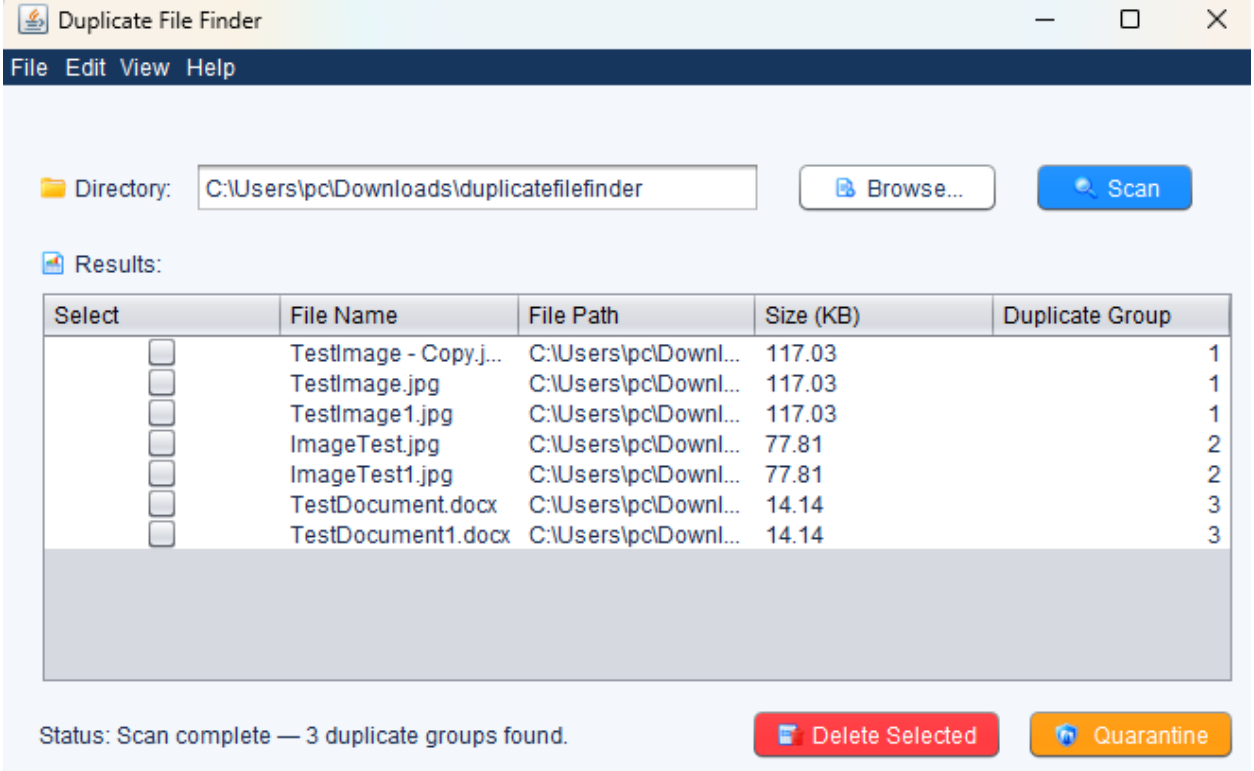
ReportGeneratorTest:

- *What it tests:* Wires all the isolated components together end-to-end.
- *Edge Cases:* Fed it bad directory paths and invalid export formats to ensure the central controller catches and rejects invalid states before they can reach the UI and crash the application.



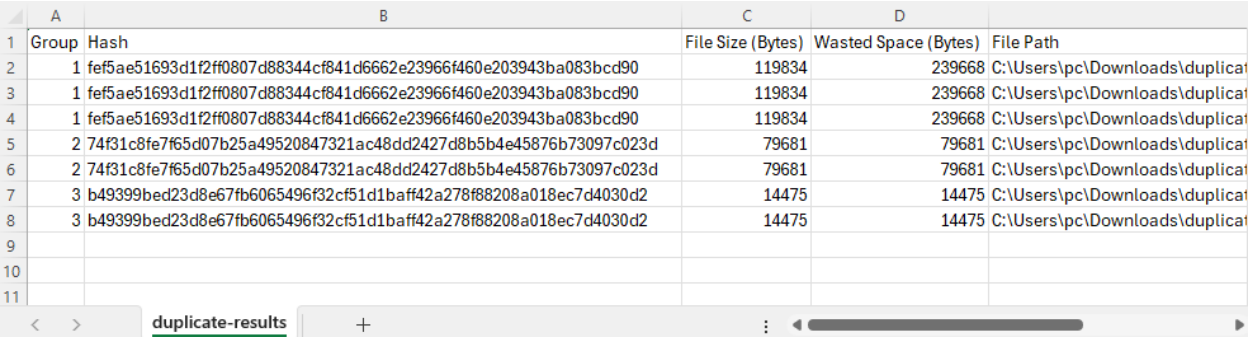
Manual Testing

1. Running a scan on duplicatefilefinder test folder created



2. Results

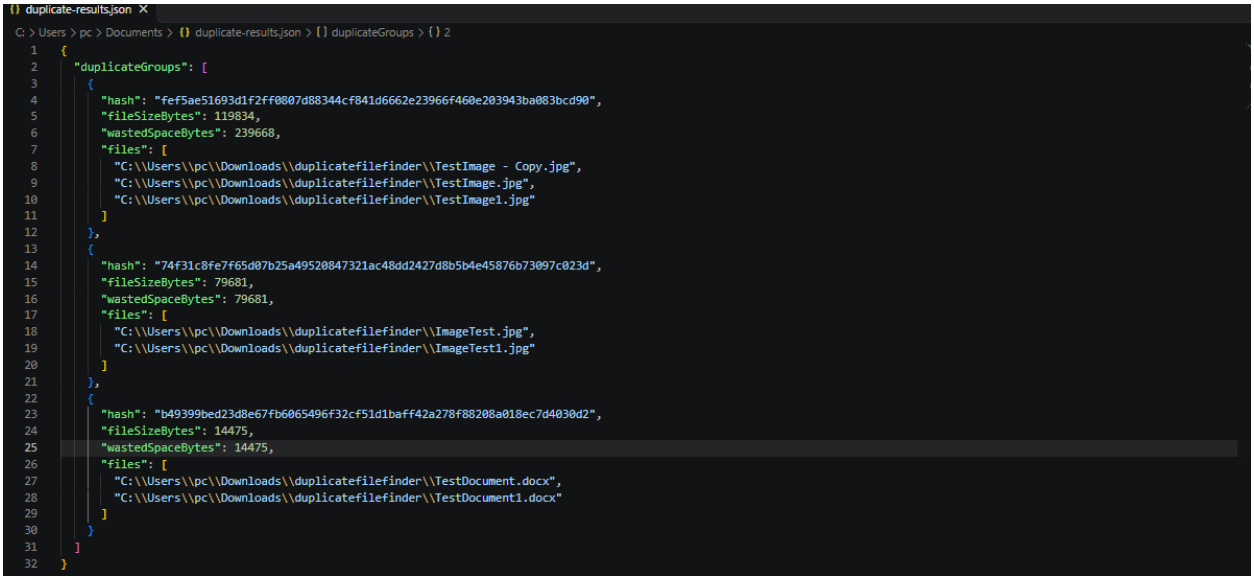
CSV scan results



	A	B	C	D	
1	Group	Hash	File Size (Bytes)	Wasted Space (Bytes)	File Path
2	1	fef5ae51693d1f2ff0807d88344cf841d6662e23966f460e203943ba083bcd90	119834	239668	C:\Users\pc\Downloads\duplicat
3	1	fef5ae51693d1f2ff0807d88344cf841d6662e23966f460e203943ba083bcd90	119834	239668	C:\Users\pc\Downloads\duplicat
4	1	fef5ae51693d1f2ff0807d88344cf841d6662e23966f460e203943ba083bcd90	119834	239668	C:\Users\pc\Downloads\duplicat
5	2	74f31c8fe7f65d07b25a4952084732 1ac48dd2427d8b5b4e45876b73097c023d	79681	79681	C:\Users\pc\Downloads\duplicat
6	2	74f31c8fe7f65d07b25a4952084732 1ac48dd2427d8b5b4e45876b73097c023d	79681	79681	C:\Users\pc\Downloads\duplicat
7	3	b49399bed23d8e67fb6065496f32cf51d1baff42a278f88208a018ec7d4030d2	14475	14475	C:\Users\pc\Downloads\duplicat
8	3	b49399bed23d8e67fb6065496f32cf51d1baff42a278f88208a018ec7d4030d2	14475	14475	C:\Users\pc\Downloads\duplicat
9					
10					
11					

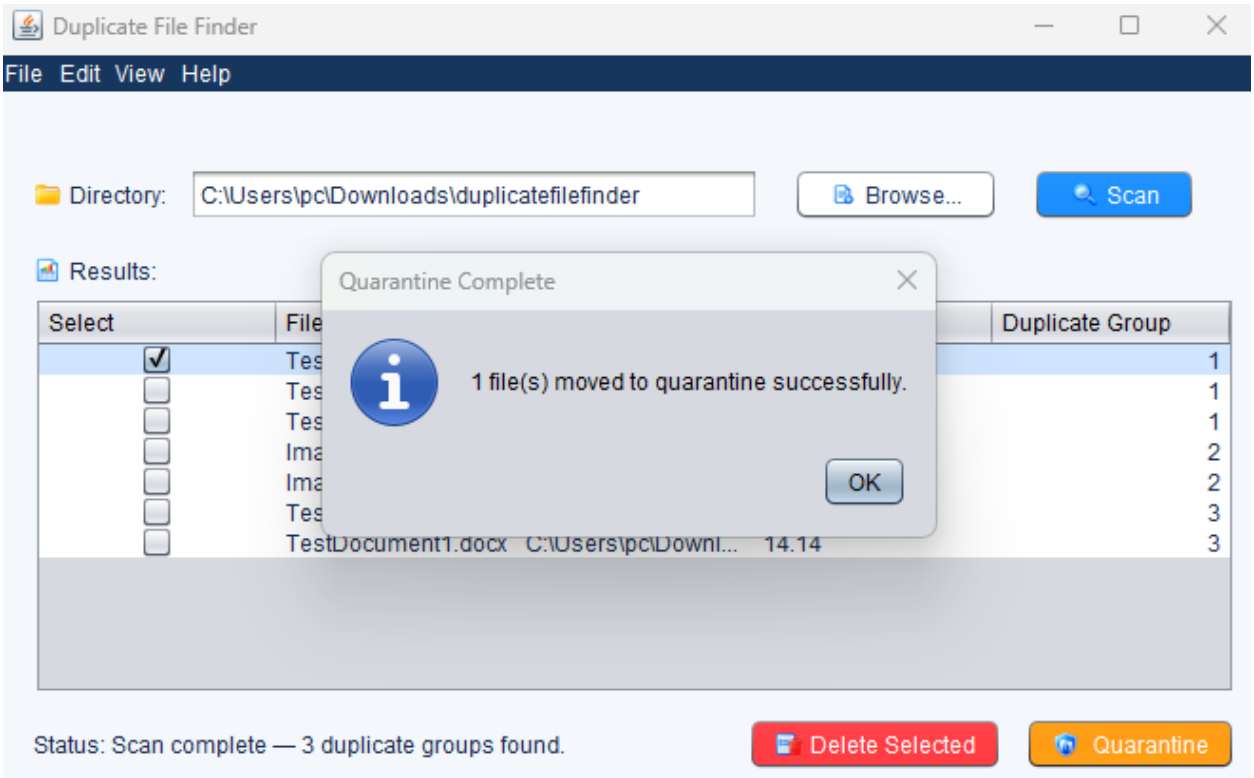
duplicate-results

JSON scan results

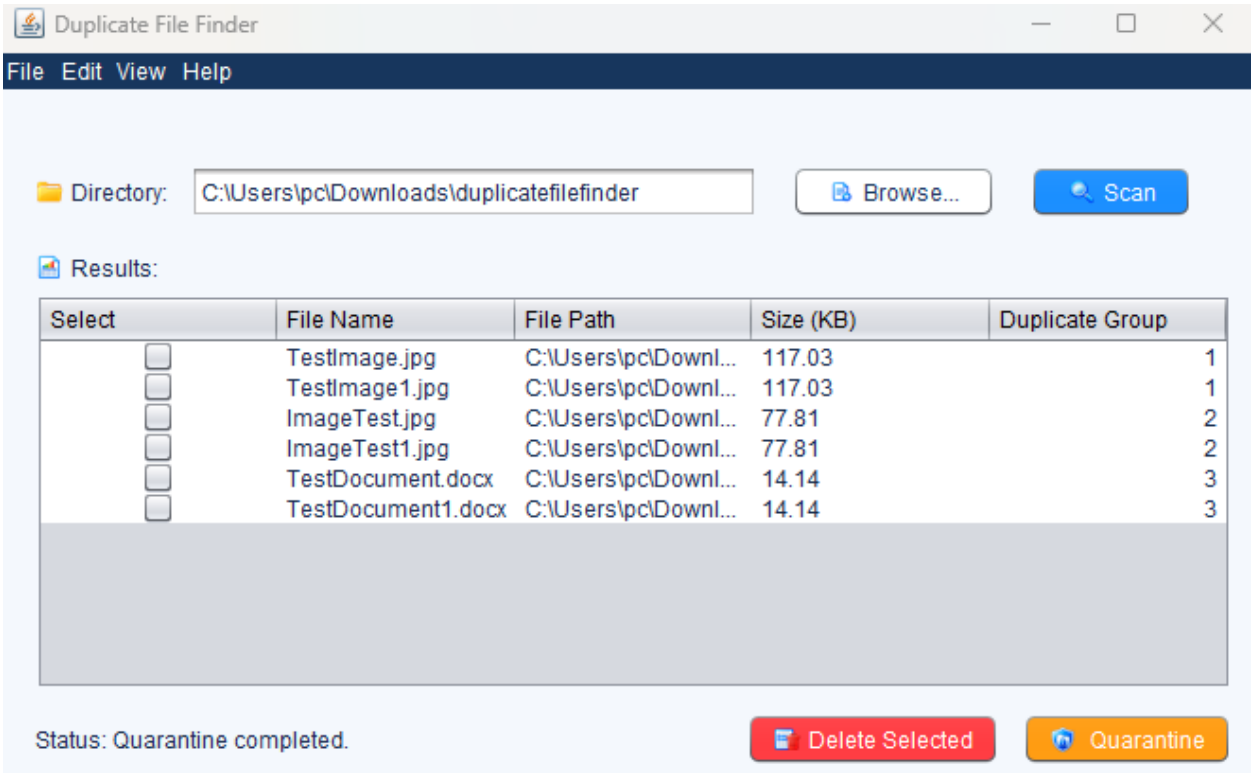


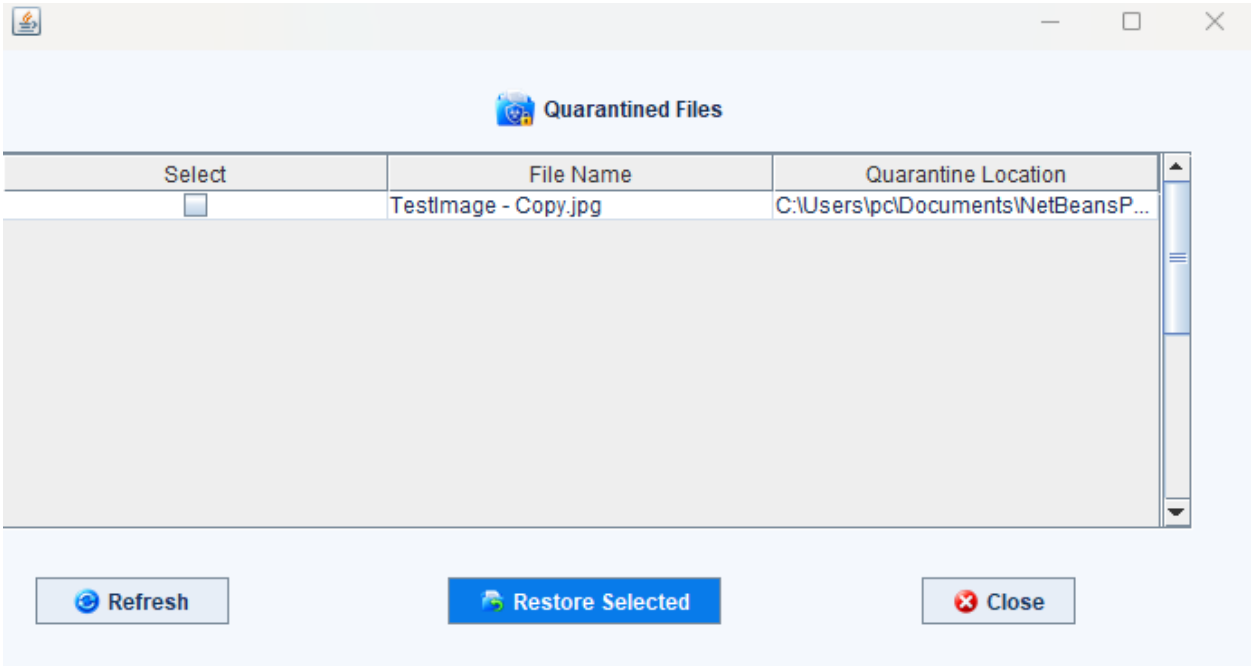
Quarantine window after refresh

- Quarantined TestImage – Copy.jpg from the scan results

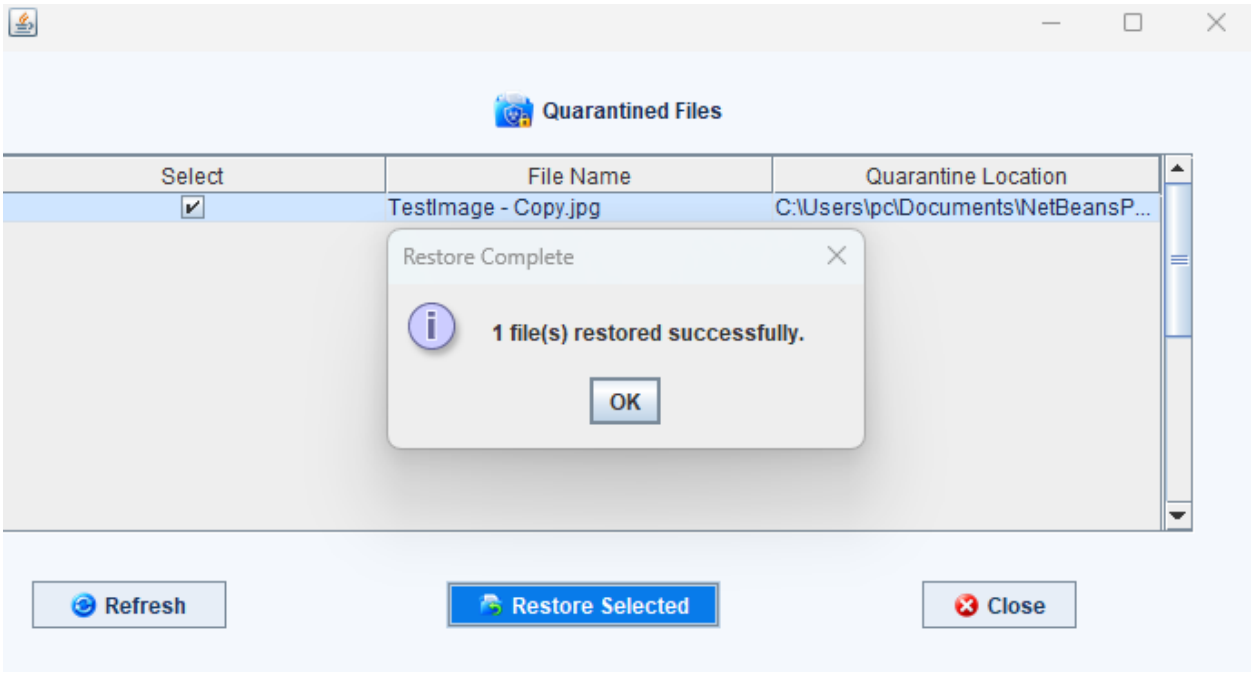


- File successfully quarantined

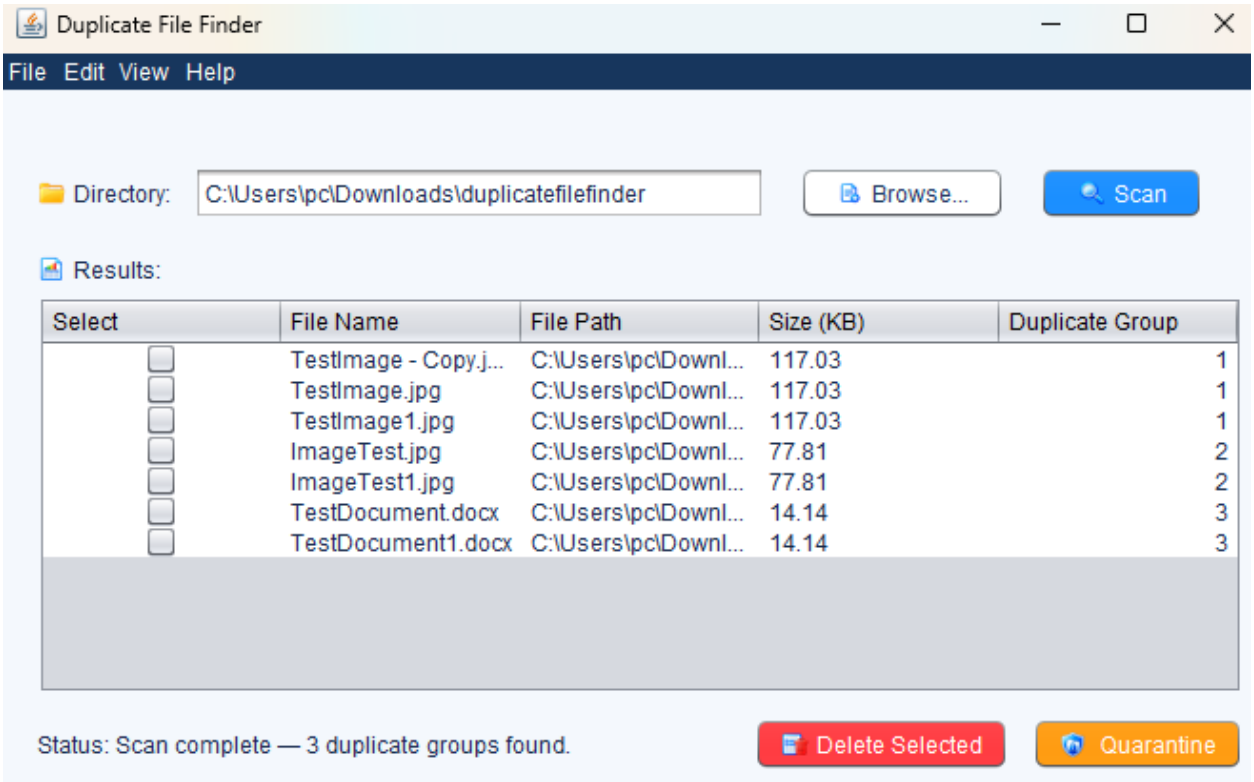




File successfully restored from quarantine back to original folder

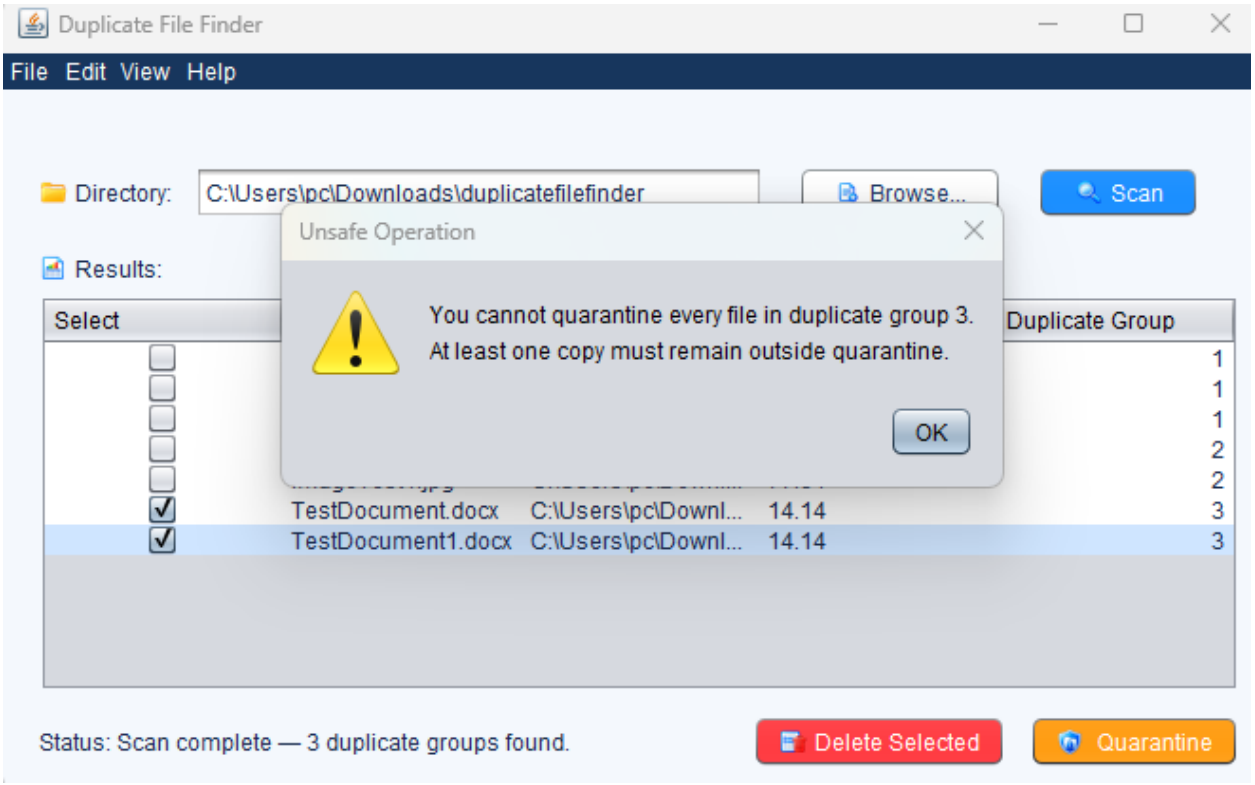


- Run a new scan (file TestImage – Copy.jpg restored)



Safety limits check:

- “Keep one copy” rule, app does not allow one to every file in duplicate group 3. At least one copy must not be quarantined and left in original folder



Test permanent delete:

- Delete TestImage – Copy.jpg permanently without quarantine

